

מבוא למדעי המחשב

שיעור 4: תכנות מונחה עצמים, איטרטורים

לקריאה נוספת:

• קרין גורן, סודות מתוקים, 2006.

- Mel Brooks, History of the World: Part I, 1981.
- Charles de Secondat, Baron de Montesquieu. *De l'Esprit des Loix*, Livre XX, 1748.
- Leonardo Pisano (Fibonacci), *Il Liber Abbaci*, 1202.
- Parmanand Singh, the so-called Fibonacci numbers in ancient and medieval India, *Historia Mathematica*, 12:229-244, 1985.

תכנות מונחה עצמים

מה זה?

- פרדיגמת תכנות שמבוססת על זיווג של נתונים וקוד
- אובייקטים מחזיקים הן נתונים והן קוד שניגש ומעבד את הנתונים
- נדגים בעזרת **מחלקה** בשם Car:
 - הטיפוס (type) הוא Car
 - מכוניות ספציפיות שנוצרות מטיפוס Car הן **אובייקטים** (מופעים - instances)

- **מחלקה:** הטיפוס + תיאור
ההתנהגות של אובייקט
מהטיפוס הזה (התבנית)
- **אובייקט:** מופע ספציפי של
המחלקה (עוגייה אחת)



שימוש ב-Car

```
car1 = Car()
```

יצירת מכונית חדשה

```
car2 = Car()
```

עוד אחת ...

```
car1.add_fuel(15)
```

פעולות על מכונית

```
car1.drive(5000)
```

```
car2.add_fuel(30)
```

שליפת נתונים על מכונית

```
print(car2.get_mileage())
```

```
print(car1)
```

0

Car{fuel:0.0, mileage:240}

שימו לב:

- כאשר משתמשים במכונית, הנתונים שהיא מחזיקה משתנים (נוסף דלק, המיקום משתנה, נצרך דלק, ...).
- המכוניות שומרות נתונים על מצבם.
- למכוניות יש פונקציות (מתודות) שמאפשרות שינוי וגישה לנתונים.
- כל מכונית היא אובייקט נפרד עם נתוני מצב נבדלים.

מחלקה מול אובייקט

```
class Car:  
    pass
```

זה מגדיר מחלקה (class) חדשה בשם Car.
מקובל ששמות מחלקות מתחילים באות גדולה.

```
car1 = Car()  
car2 = Car()
```

פקודות נוספות תופענה כאן

הפקודות הללו מייצרות שני אובייקטים מטיפוס Car.
שימו לב ל-().

שדות

```
class Car:  
    pass
```

```
car1 = Car()  
car2 = Car()
```

```
car1.fuel = 10  
car2.fuel = 20  
  
print(car1.fuel)
```

כאן מוסיפים לשני האובייקטים car1 ו-car2 שדה (נתון) שמחזיק את כמות הדלק במיכל

שדות כאלה נקראים:
member variables או member fields או attributes

איתחול אובייקטים

```
class Car:  
    def __init__(self, fuel=0):  
        self.fuel = fuel  
        self.mileage = 0
```

```
car1 = Car(10)  
car2 = Car()
```

```
print(car1.fuel)
```

מתודת האיתחול.
מתודה בשם `__init__` מופעלת בכל פעם שיוצרים אובייקט חדש מהמחלקה.
נקראת גם **יוצר** (constructor)

`self` היא מילה שמורה של פייתון - מראה המקום של האובייקט החדש שנוצר.

דוגמה להעברת פרמטר ל-`__init__`

המצב של אובייקט

- הנתונים שאובייקט מחזיק הם **המצב** (state) שלו.
- המצב של האובייקט יכול להשתנות כתוצאה מביצוע פעולות עליו.
- מבצעים פעולות על האובייקט על ידי הפעלת מתודות (methods) שלו.
- מתודה היא פונקציה שפועלת על האובייקט ויכולה לשנות את מצבו.

מתודות

```
class Car:
```

```
def __init__(self, fuel=0, km_per_litre=16):  
    self.__fuel = max(fuel, 0)  
    self.__mileage = 0  
    self.__km_p_litre = max(km_per_litre, 1)
```

```
def add_fuel(self, amount):  
    if amount > 0:  
        self.__fuel += amount
```

```
def drive(self, distance):  
    if distance > 0:  
        driving_distance = min(distance,  
                                self.__fuel*self.__km_p_litre)  
        self.__mileage += driving_distance  
        self.__fuel -= driving_distance/self.__km_p_litre
```

```
def get_fuel(self):  
    return self.__fuel
```

```
def get_mileage(self):  
    return self.__mileage
```

פונקציות המוגדרות בתוך
המחלקה נקראות מתודות
(methods) (או member
functions).

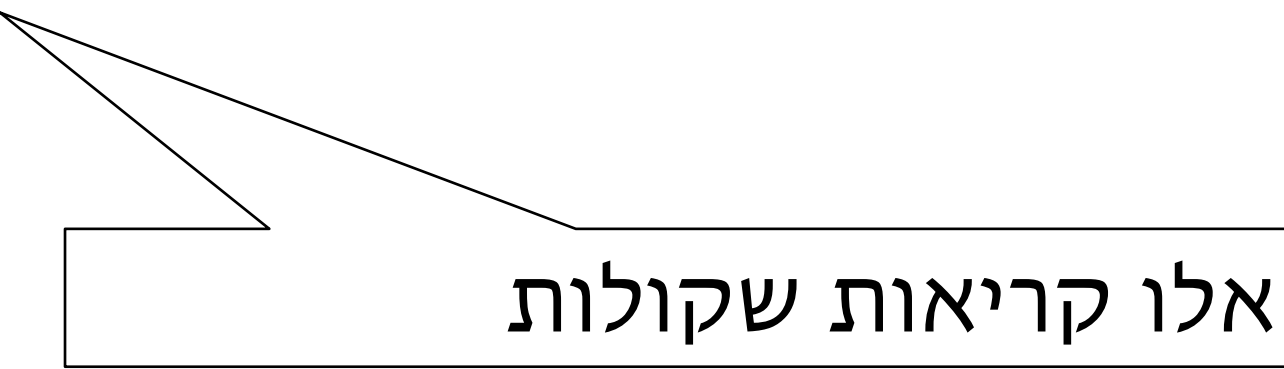
הן מוסיפות פונקציונליות
לאובייקטים מהמחלקה.

הפרמטר הראשון הוא self,
שזה האובייקט שהמתודה
מופעלת עליו.

```
car1 = Car()  
car2 = Car()  
car1.add_fuel(15)  
car1.drive(5000)  
car2.add_fuel(30)  
print(car2.get_mileage())
```

דוגמה לשימוש

```
car1.add_fuel(15)  
Car.add_fuel(car1, 15)
```



אלו קריאות שקולות

החבאת שדות

- תכנות מונחה עצמים טוב מבטיח שהאובייקטים ינוהלו בעקביות.
- לדוגמה: הסעת המכונית מחייבת הגדלת הקילומטרז' וצריכת דלק.
- אם המתכנת ניגש ישירות לשדה, זה עלול ליצור חוסר עקביות. למשל, פקודה חיצונית `-2 = car1.__mileage`.
- על כן נהוג לכתוב מחלקות שמחביאות את המימוש הפנימי ומאפשרות שינוי מצב רק דרך מתודות ממשק.
- אם המחלקה ממומשת היטב, יהיה בלתי אפשרי לשנות את המצב למשהו לא עקבי דרך הממשק.

```
class Car:
```

```
    def __init__(self, fuel=0, km_per_litre=16):
```

```
        self.__fuel = max(fuel, 0)
```

```
        self.__mileage = 0
```

```
        self.__km_p_litre = max(km_per_litre, 1)
```

```
    def add_fuel(self, amount):
```

```
        if amount > 0:
```

```
            self.__fuel += amount
```

```
    def drive(self, distance):
```

```
        if distance > 0:
```

```
            driving_distance = min(distance,
```

```
                                   self.__fuel*self.__km_p_litre)
```

```
            self.__mileage += driving_distance
```

```
            self.__fuel -= driving_distance/self.__km_p_litre
```

```
    def get_fuel(self):
```

```
        return self.__fuel
```

שדות פנימיים (אסורי
גישה חיצונית) מכונים
בשם שמתחיל ב-__

ניגשים לנתוני מצב
כאלה רק דרך מתודות
ממשק

```
car1 = Car()  
print(car1.get_fuel())  
print(car1.__fuel)
```

פייתון חוסם (במידת מה) גישה
כזו-השדה עדיין נגיש, אבל השם
שלו הוחלף ל-Car__fuel.

ממשק אפליקציה (API) ותיכון על פי חוזה

- המתודות של מחלקה מגדירות את ה-API שלה – האופן שבו מתכנתים מתקשרים עם אובייקטים מהמחלקה הזו.
- הפרדיגמה של **תיכון על פי חוזה** (design by contract):
 - ה-API מתחייב רק על ההתנהגות של האובייקטים, לא על המימוש הפנימי.
 - **כימוס** (encapsulation): מתכנת שמשתמש במחלקה זקוק רק להגדרת הממשק; לא נדרש לפרטים הפנימיים.

תיעוד מחלקות

- כיתבו docstring בכל מתודה של המחלקה שמתעד את הממשק – מה המתודה דורשת מהמשתמש בה ומה היא מבטיחה.
- כיתבו doctoring בתחילת המחלקה שמסביר מהו אובייקט כזה – זה המקום לפרט שדות שהם פומביים (לא פנימיים), אם יש כאלה.

```
class Cat:
    """ this class represents a kitty cat

    name: the name of the cat

    """

    def __init__(self, name):
        """
        creates a cat with the given name
        """
        self.name = name
```

משתני מחלקה

```
class Car:
    def __init__(self, fuel_amount):
        self.fuel = fuel_amount
```

```
NUM_WHEELS = 4
```

```
car1 = Car(3)
car2 = Car(5)
```

```
print("car1.fuel:", car1.fuel)
print("car2.fuel:", car2.fuel)
```

```
print("car1.NUM_WHEELS:", car1.NUM_WHEELS)
print("car2.NUM_WHEELS:", car2.NUM_WHEELS)
print("Car.NUM_WHEELS:", Car.NUM_WHEELS)
```

משתני מחלקה מוגדרים בבלוק של המחלקה כולה. הם גלובליים לכל האובייקטים מהמחלקה. (לא נוצר עותק נפרד לכל אובייקט).

הפלט:

```
car1.fuel: 3
car2.fuel: 5
car1.NUM_WHEELS: 4
car2.NUM_WHEELS: 4
Car.NUM_WHEELS: 4
```

העולם על פי פייתון הוא אובייקטים ...

```
x = 30
print(x.real)
print(x.bit_length())

lst = ['a', 'b', 'c', 'a']
print(lst.count('a'))
lst.remove('b')
lst.insert(1, 'h')
lst.sort()
print(lst)
```

```
30
5
2
['a', 'a', 'c', 'h']
```

אובייקטים הם mutable

```
def my_func(some_car):  
    some_car.add_fuel(20)
```

מה קורה כאן?

```
my_car = Car(10)  
print(my_car.get_fuel())  
my_func(my_car)  
print(my_car.get_fuel())
```

המשתנה my_car
הוא מראה מקום

- כך בודקים אם אובייקט הוא מטיפוס נתון:

```
print(isinstance(car1, Car))  
print(type(car1))  
print(type(car1) == Car)
```

True

<class '__main__.Car'>

True

איטרטורים

איטרטורים

- בפייתון, מבנה שנקרא איטרטור (iterator) מאפשר לנו להגדיר סדרות ערכים כרצוננו.
- iterator הוא אובייקט שהממשק שלו כולל את שתי המתודות הבאות:
 1. מתודה `__iter__(self)` שמחזירה את `self`.
 2. מתודה `__next__(self)` שמחזירה את הערך הבא בסדרה, או מעלה את הפסיקה `StopIteration` אם הסדרה הסתיימה.
- iterable הוא אובייקט שהממשק שלו כולל את המתודה `__iter__(self)` שמחזירה אובייקט שהוא iterator.
- שימו לב שכל iterator הוא גם iterable.

דוגמה: סדרת פיבונצ'י

- הסדרה מוגדרת באינדוקציה:

$$F_1 = F_2 = 1$$

$$F_n = F_{n-2} + F_{n-1}$$

- האברים הראשונים בסדרה: 1, 1, 2, 3, 5, 8, 13, 21, ...

דוגמה: סדרת פיבונצ'י

- הסדרה מוגדרת באינדוקציה:

$$F_1 = F_2 = 1$$

$$F_n = F_{n-2} + F_{n-1}$$

- האברים הראשונים בסדרה: 1, 1, 2, 3, 5, 8, 13, 21, ...

זוג ארנבים מזדווגים מגיל חודש.
ההריון נמשך חודש,
בסופו נולד זוג נוסף.
מתחילים מזוג אחד בגיל 0.
ארנבים לא מתים אף פעם.
כמה זוגות יהיו אחרי n חודשים?

דוגמה: סדרת פיבונצ'י

• הסדרה מוגדרת באינדוקציה:

$$F_1 = F_2 = 1$$

$$F_n = F_{n-2} + F_{n-1}$$

• האברים הראשונים בסדרה: 1, 1, 2, 3, 5, 8, 13, 21, ...

הערה: הסדרה היתה ידועה למתמטיקאים
הודים לפני פיבונצ'י:

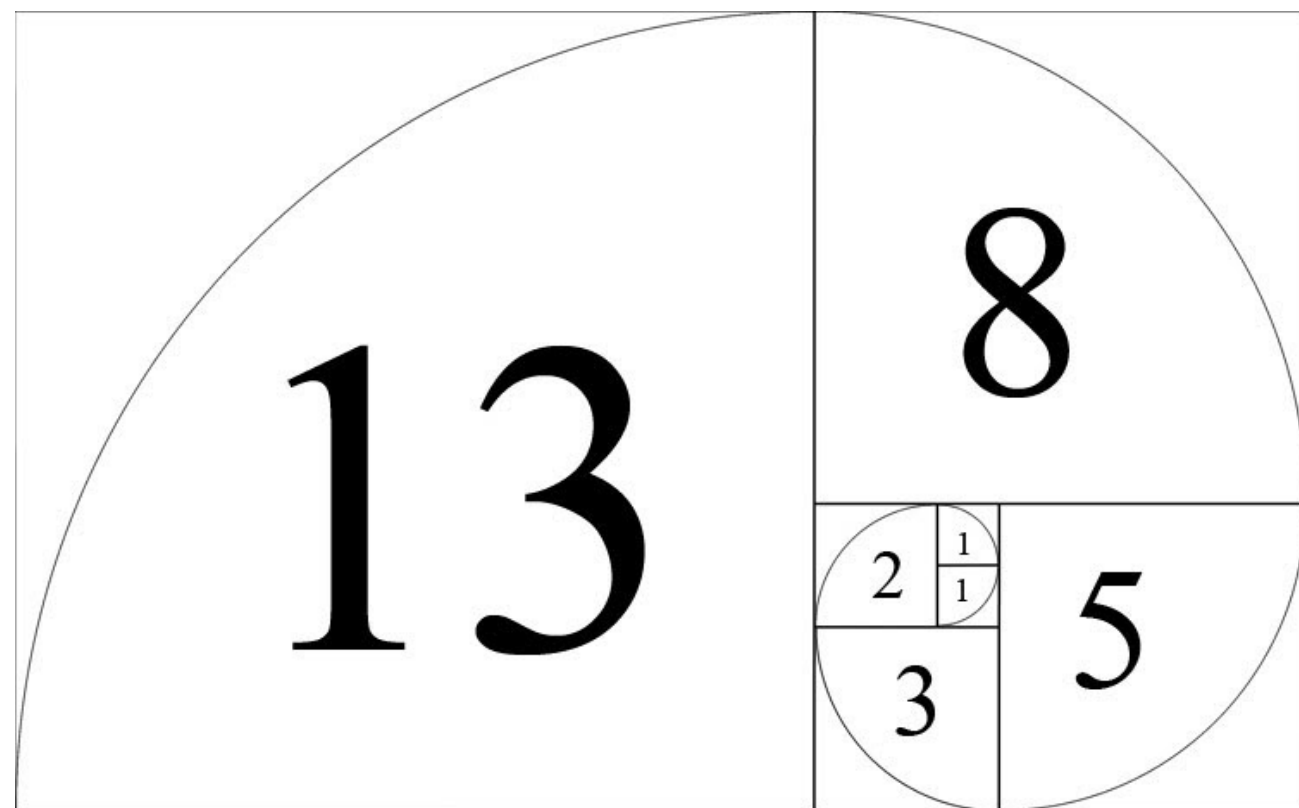
पिङ्गल - Pingala, c.200 B.C.

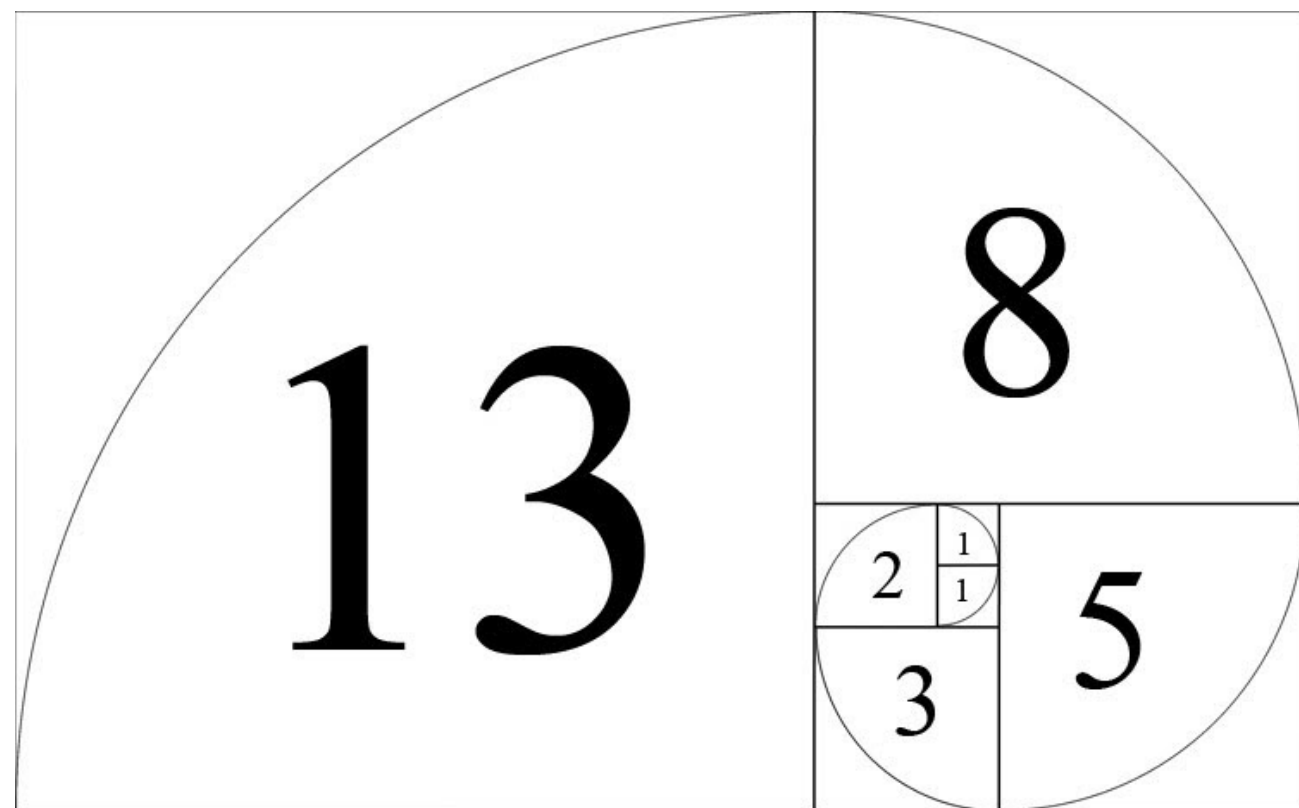
विरहार्क - Virhanka, c.700

गोपाला - Gopala, before 1135

हेमचन्द्र सूरी - Acharya Hemachandra, c.1150

זוג ארנבים מזדווגים מגיל חודש.
ההריון נמשך חודש,
בסופו נולד זוג נוסף.
מתחילים מזוג אחד בגיל 0.
ארנבים לא מתים אף פעם.
כמה זוגות יהיו אחרי n חודשים?





מתוך: inspirationgreen.com

איטרטור סדרת פיבונצ'י

Python 3.4.1: FibonacciIterator.py - /Users/yrabani/Documents/10 פיתרון/תשעה/שיעור 10/Code/FibonacciIterator.py

```
class FibonacciIterator:
    '''
    An object of this class is an iterator of the Fibonacci sequence
    1, 1, 2, 3, 5, 8, 13, ...
    The sequence terminates after the last entry which is < n.
    '''

    def __init__(self, n):
        assert type(n) is int
        self._n = n          # marks the end of the sequence
        self._prev = 0       # previous entry
        self._curr = 1       # current entry

    def __iter__(self):
        return self

    def __next__(self):
        if (self._curr < self._n):
            curr = self._curr          # put aside the value to be returned
            self._curr += self._prev   # compute the next entry in the sequence
            self._prev = curr          # returned value will be the previous
            return curr
        else:
            raise StopIteration()
```

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)

[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin

Type "copyright", "credits" or "license()" for more information.

>>> ===== RESTART =====

>>>

```
>>> for i in FibonacciIterator(20):  
    print(i)
```

1
1
2
3
5
8
13

```
>>> fibonacci = FibonacciIterator(20)  
>>> for i in fibonacci:  
    print(i)
```

1
1
2
3
5
8
13

```
>>> for i in fibonacci:  
    print(i)
```

>>>

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)

[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin

Type "copyright", "credits" or "license()" for more information.

>>> ===== RESTART =====

>>>

```
>>> for i in FibonacciIterator(20):
    print(i)
```

1

1

2

3

5

8

13

```
>>> fibonacci = FibonacciIterator(20)
```

```
>>> for i in fibonacci:
    print(i)
```

1

1

2

3

5

8

13

```
>>> for i in fibonacci:
    print(i)
```

>>>

- לולאת for מתחילה בהפעלת המתודה `__iter__` כדי לקבל איטרטור.
- בכל איטרציה היא מפעילה את המתודה `__next__` כדי לקבל את הערך הבא של `i`.
- הפסיקה `StopIteration` מציינת עבור הלולאה את סיומה.

```

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
>>> for i in FibonacciIterator(20):
        print(i)

1
1
2
3
5
8
13
>>> fibonacci = FibonacciIterator(20)
>>> for i in fibonacci:
        print(i)

1
1
2
3
5
8
13
>>> for i in fibonacci:
        print(i)

>>>

```

- לולאת for מתחילה בהפעלת המתודה `__iter__` כדי לקבל איטרטור.
- בכל איטרציה היא מפעילה את המתודה `__next__` כדי לקבל את הערך הבא של `i`.
- הפסיקה `StopIteration` מציינת עבור הלולאה את סיומה.

- האובייקט `fibonacci` הוא `iterator`, לכן `__iter__` מחזירה את `self`.
- אם הגענו לסוף הסדרה, כל קריאה נוספת ל-`__next__` גורמת להעלאת הפסיקה `StopIteration`.

```

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
>>> for i in FibonacciIterator(20):
        print(i)

1
1
2
3
5
8
13
>>> fibonacci = FibonacciIterator(20)
>>> for i in fibonacci:
        print(i)

1
1
2
3
5
8
13
>>> for i in fibonacci:
        print(i)

```

אפשר לפתור את הבעייה הזו
 על ידי שימוש ב-iterable.

- לולאת for מתחילה בהפעלת המתודה `__iter__` כדי לקבל איטרטור.
- בכל איטרציה היא מפעילה את המתודה `__next__` כדי לקבל את הערך הבא של `i`.
- הפסיקה `StopIteration` מציינת עבור הלולאה את סיומה.

- האובייקט `fibonacci` הוא `iterator`, לכן `__iter__` מחזירה את `self`.
- אם הגענו לסוף הסדרה, כל קריאה נוספת ל-`__next__` גורמת להעלאת הפסיקה `StopIteration`.

דוגמה משופרת

Python 3.4.1: Fibonacciterable.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור 10/Code/Fibonacciterabl...

```
class FibonacciIterable:
    """
    An object of this class is an iterable that returns an
    iterator for the Fibonacci sequence 1, 1, 2, 3, 5, 8, 13, ...
    The initialization parameter n marks the end of the sequence:
    the last entry is the largest in the sequence which is < n.
    """

    def __init__(self, n):
        assert type(n) is int
        self._n = n          # marks the end of the sequence

    def __iter__(self):
        return FibonacciIterator(self._n)
```

Ln: 6 Col: 63

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)

[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin

Type "copyright", "credits" or "license()" for more information.

>>> ===== RESTART =====

>>>

>>> fibonacci = FibonacciIterable(20)

>>> for i in fibonacci:
 print(i)

1

1

2

3

5

8

13

>>> for i in fibonacci:
 print(i)

1

1

2

3

5

8

13

>>>

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
```

```
>>> ===== RESTART =====
```

```
>>>
```

```
>>> fibonacci = FibonacciIterable(20)
```

```
>>> for i in fibonacci:
    print(i)
```

```
1
```

```
1
```

```
2
```

```
3
```

```
5
```

```
8
```

```
13
```

```
>>> for i in fibonacci:
    print(i)
```

```
1
```

```
1
```

```
2
```

```
3
```

```
5
```

```
8
```

```
13
```

```
>>>
```

- הטיפוסים range, list, tuple, str הם iterable, אבל הם לא האיטרטורים של עצמם.

- לכן, לדוגמה, אפשר לעבור על אותו list פעמיים בשתי לולאות עוקבות.

גנרטורים

- גנרטור (generator) הוא צורה מקוצרת לייצר איטרטור.
- גנרטור נראה בדיוק כמו פונקציה, אלא שבמקום פקודות return יש בו פקודות yield.
- קריאה לגנרטור תוך העברת הפרמטרים הנדרשים מייצרת איטרטור.
- כדי להבין מה האיטרטור עושה, נבצע את התרגיל המחשבתי הבא:
 - נריץ בדמיוננו את הקוד של הגנרטור.
 - בכל פעם שנגיע לפקודת yield פשוט נרשום את ערך הביטוי שמופיע שם ונמשיך בהרצה הדמיונית.
 - סדרת הערכים שרשמנו היא הסדרה שמחזיר האיטרטור שנוצר.
- כל קריאה לגנרטור מייצרת את האיטרטור מחדש.

דוגמה

Python 3.4.1: fibonacci_generator.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור 10/Code/fibonacci_ge...

```
def fibonacci_generator(n):  
    '''  
    This generator iterates over the Fibonacci sequence  
    1, 1, 2, 3, 5, 8, 13, ...  
    The sequence terminates after the last entry which is < n  
    '''  
    prev = 0  
    curr = 1  
    while curr < n:  
        ret = curr  
        curr += prev  
        prev = ret  
        yield ret
```

Ln: 13 Col: 17

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)

[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin

Type "copyright", "credits" or "license()" for more information.

>>> ===== RESTART =====

>>>

```
>>> for i in fibonacci_generator(20):  
    print(i)
```

1

1

2

3

5

8

13

>>>

פעולות עם איטרטורים

- מיפוי (map): רוצים סדרה שנוצרת על ידי הפעלת פונקציה על איברי סדרה נתונה.
- ניפוי (filter): רוצים תת-סדרה של סדרה נתונה.
- צמצום (reduce): חישוב ערך בודד שהוא פונקציה של כל הסדרה.
- בפייתון יש כלים כלליים לעבוד עם איטרטורים וגם כלים לבצע פעולות ספציפיות (במודול itertools).

דוגמאות

```
Python 3.4.1 Shell

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> from functools import reduce
>>> PRIME = 13
>>> squares = list(map(lambda x: x**2 % PRIME, range(PRIME)))
>>> print(squares)
[0, 1, 4, 9, 3, 12, 10, 10, 12, 3, 9, 4, 1]
>>> quadratic_residues = list(filter(lambda x: x in squares, range(PRIME)))
>>> print(quadratic_residues)
[0, 1, 3, 4, 9, 10, 12]
>>> prod = reduce(lambda x, y: x*y, filter(lambda x: x != 0, quadratic_residues))
>>> print(prod)
12960
>>> print(prod % PRIME)
12
>>>
```

Ln: 17 Col: 4

```
def my_map(func, iterable):  
    '''  
    This generates a mapped sequence.  
    "func" is a one-argument function. It must return a value (of any type).  
    "iterable" is an iterable object.  
    type(func's argument) == type(an item from iterable.__iter__())  
    '''  
    for item in iterable:  
        yield func(item)  
  
def my_filter(pred, iterable):  
    '''  
    This generates a filtered sequence.  
    "pred" is a one-argument function that returns a boolean value.  
    "iterable" is an iterable object.  
    type(func's argument) == type(an item from iterable.__iter__())  
    '''  
    for item in iterable:  
        if pred(item):  
            yield item  
  
def my_reduce(func, iterable):  
    '''  
    This reduces a sequence to a single cumulative value.  
    "func" is a two-argument function that returns a value.  
    "iterable" is an iterable object. The iterator must not return an item None.  
    type(func's arguments & return value) == type(item from iterable.__iter__())  
    '''  
    prev = None  
    for item in iterable:  
        if prev == None:  
            prev = item  
        else:  
            prev = func(prev, item)  
    return prev
```

עוד דוגמאות

Python 3.4.1: using_iter.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור 10 - פייטון/מחשב - Code/using_iter.py

```
list_of_squares = [x**2 for x in range(10)]
gen_of_squares = (x**2 for x in range(10))

print(list_of_squares)
print(gen_of_squares)

dirty = 'Go ahead. Make my day.'
clean_list = [dirty[i] for i in range(len(dirty))
              if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ']
clean_gen = (dirty[i] for i in range(len(dirty))
            if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')
clean_str = ''.join(dirty[i] for i in range(len(dirty))
                   if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')

print(dirty)
print(clean_list)
print(clean_gen)
print(clean_str)
```

Ln: 5 Col: 9

עוד דוגמאות

Python 3.4.1: using_iter.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור 10 - פייטון/הוראה/מבוא למדעי המחשב - Code/using_iter.py

```
list_of_squares = [x**2 for x in range(10)]
gen_of_squares = (x**2 for x in range(10))
```

```
print(list_of_squares)
print(gen_of_squares)
```

list comprehension
(מייצר list)

```
dirty = 'Go ahead. Make my day.'
clean_list = [dirty[i] for i in range(len(dirty))
               if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ']
clean_gen = (dirty[i] for i in range(len(dirty))
             if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')
clean_str = ''.join(dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')
print(dirty)
print(clean_list)
print(clean_gen)
print(clean_str)
```

Ln: 5 Col: 9

עוד דוגמאות

Python 3.4.1: using_iter.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור 10 - המחשב - Code/using_iter.py

```
list_of_squares = [x**2 for x in range(10)]
gen_of_squares = (x**2 for x in range(10))

print(list_of_squares)
print(gen_of_squares)

dirty = 'Go ahead. Make my day.'
clean_list = [dirty[i] for i in range(len(dirty))
               if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ']
clean_gen = (dirty[i] for i in range(len(dirty))
              if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')
clean_str = ''.join(dirty[i] for i in range(len(dirty))
                     if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')

print(dirty)
print(clean_list)
print(clean_gen)
print(clean_str)
```

generator expression
(מייצר גנרטור)

Ln: 5 Col: 9

עוד דוגמאות

Python 3.4.1: using_iter.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור 10 - פיתרון/Code/using_iter.py

```
list_of_squares = [x**2 for x in range(10)]
gen_of_squares = (x**2 for x in range(10))

print(list_of_squares)
print(gen_of_squares)

dirty = 'Go ahead. Make my day.'
clean_list = [dirty[i] for i in range(len(dirty))
              if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ']
clean_gen = (dirty[i] for i in range(len(dirty))
            if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')
clean_str = ''.join(dirty[i] for i in range(len(dirty))
                   if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')

print(dirty)
print(clean_list)
print(clean_gen)
print(clean_str)
```

list comprehension
בתוספת פילטר

Ln: 5 Col: 9

עוד דוגמאות

Python 3.4.1: using_iter.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור - Code/using_iter.py

```
list_of_squares = [x**2 for x in range(10)]
gen_of_squares = (x**2 for x in range(10))

print(list_of_squares)
print(gen_of_squares)

dirty = 'Go ahead. Make my day.'
clean_list = [dirty[i] for i in range(len(dirty))
               if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ']
clean_gen = (dirty[i] for i in range(len(dirty))
             if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')
clean_str = ''.join(dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i==0 or dirty[i-1] != ' ')

print(dirty)
print(clean_list)
```

זו מתודה של המחלקה `str`.
היא מקבלת אובייקט `iterable` שהאיטרטור שלו מפיק מחרוזות.
הקריאה `string.join(iterable)` משרשרת את סדרת המחרוזות
שהאיטרטור של `iterable` מפיק, מופרדות על ידי `string`.

עוד דוגמאות

Python 3.4.1: using_iter.py - /Users/ywabani/Documents/10 פייתון/תשעה/שיעור - Code/using_iter.py

```
list_of_squares = [x**2 for x in range(10)]
gen_of_squares = (x**2 for x in range(10))

print(list_of_squares)
print(gen_of_squares)

dirty = 'Go ahead. Make my day.'
clean_list = [dirty[i] for i in range(len(dirty))
              if dirty[i]!=' ' or i==0 or dirty[i-1]!=' ']
clean_gen = (dirty[i] for i in range(len(dirty))
            if dirty[i]!=' ' or i==0 or dirty[i-1]!=' ')
clean_str = ''.join(dirty[i] for i in range(len(dirty))
                   if dirty[i]!=' ' or i==0 or dirty[i-1]!=' ')

print(dirty)
print(clean_list)
print(clean_gen)
print(clean_str)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
<generator object <genexpr> at 0x105873f30>
Go ahead. Make my day.
['G', 'o', ' ', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.']
<generator object <genexpr> at 0x10587cbd0>
Go ahead. Make my day.
```

פונקציות מ- itertools

Python 3.4.1: itertools_demo.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור - Code/itertools_demo.py

```
from itertools import accumulate, chain

# Compute the partial sums of an iterator
# accumulate(iterable) or accumulate(iterable, function)
#
partial_sums = accumulate(range(1,6))
print(' ')
print('partial sums demo:')
print(partial_sums)
print(list(partial_sums))

# Chain together several iterators
# chain(iterable, iterable, ...)
#
sent = chain('Ou il y a du commerce', '\n', 'Il y a des mœurs douces.')
print(' ')
print('chain demo:')
print(sent)
print(''.join(sent))
```

פונקציות מ- itertools

Python 3.4.1: itertools_demo.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור 10/Code/itertools_demo.py

```
from itertools import accumulate, chain

# Compute the partial sums of an iterator
# accumulate(iterable) or accumulate(iterable, function)
#
partial_sums = accumulate(range(1, 6))
print(' ')
print('partial sums demo:')
print(partial_sums)
print(list(partial_sums))

# Chain together several iterators
# chain(iterable, iterable, ...)
#
sent = chain('Ou il y a du commerce', '\n', 'Il y a des mœurs douces.')
print(' ')
print('chain demo:')
print(sent)
print(''.join(sent))
```

```
partial sums demo:
<itertools.accumulate object at 0x103695508>
[1, 3, 6, 10, 15]
```

פונקציות מ- itertools

Python 3.4.1: itertools_demo.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור - Code/itertools_demo.py

```
from itertools import accumulate, chain

# Compute the partial sums of an iterator
# accumulate(iterable) or accumulate(iterable, function)
#
partial_sums = accumulate(range(1,6))
print(' ')
print('partial sums demo:')
print(partial_sums)
print(list(partial_sums))

# Chain together several iterators
# chain(iterable, iterable, ...)
#
sent = chain('Ou il y a du commerce', '\n', 'Il y a des mœurs douces.')
print(' ')
print('chain demo:')
print(sent)
print(''.join(sent))
```

פונקציות מ- itertools

Python 3.4.1: itertools_demo.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור/Code/itertools_demo.py

```
from itertools import accumulate, chain

# Compute the partial sums of an iterator
# accumulate(iterable) or accumulate(iterable, function)
#
partial_sums = accumulate(range(1,6))
print(' ')
print('partial sums demo:')
print(partial_sums)
print(list(partial_sums))

# Chain together several iterators
# chain(iterable, iterable, ...)
#
sent = chain('Ou il y a du commerce', '\n', 'Il y a des mœurs douces.')
print(' ')
print('chain demo:')
print(sent)
print(''.join(sent))
```

```
chain demo:
<itertools.chain object at 0x103597470>
Ou il y a du commerce
Il y a des mœurs douces.
```

פונקציות מ- itertools

Python 3.4.1: itertools_demo.py - /Users/yrabani/Documents/10 פייתון/תשעה/שיעור - Code/itertools_demo.py

```
from itertools import accumulate, chain

# Compute the partial sums of an iterator
# accumulate(iterable) or accumulate(iterable, function)
#
partial_sums = accumulate(range(1,6))
print(' ')
print('partial sums demo:')
print(partial_sums)
print(list(partial_sums))

# Chain together several iterators
# chain(iterable, iterable, ...)
#
sent = chain('Ou il y a du commerce', '\n', 'Il y a des mœurs douces.')
print(' ')
print('chain demo:')
print(sent)
print(''.join(sent))
```

עוד על מחלקות

פולימורפיזם

לעיתים רוצים להפעיל אותה פעולה על אובייקטים מטיפוסים שונים.

```
class Cat:
    def make_sound(self):
        print("Meow!")
```

```
class Dog:
    def make_sound(self):
        print("Woof Woof!")
```

```
some_animals = [Cat(), Dog(), Dog()]
for animal in some_animals:
    animal.make_sound() ←
```

(יש שיטות טובות יותר ב-OOP לממש את הדוגמה הזו)

עוד דוגמה

```
class Point2D:
    def __init__(self,x=0,y=0):
        self.__x = x
        self.__y = y

    def translate(self,dx,dy):
        self.__x += dx
        self.__y += dy

    def distance_from(self,other):
        dx = self.__x - other.__x
        dy = self.__y - other.__y
        return (dx**2+dy**2)**(1/2)

    def copy(self):
        return Point2D(self.__x,self.__y)

    def get_x(self):
        return self.__x

    def get_y(self):
        return self.__y
```

```
class Rectangle2D:
    def __init__(self, corner, width, height):
        self.__corner = corner.copy()
        self.__width = width
        self.__height = height

    def get_area(self):
        return self.__width*self.__height

    def translate(self, dx, dy):
        self.__corner.translate(dx, dy)

    def get_midpoint(self):
        return Point2D(self.__corner.get_x()+self.__width/2,
                        self.__corner.get_y()+self.__height/2)

    def copy(self):
        return Rectangle2D(self.__corner.copy(),
                            self.__width, self.__height)
```

שימוש

```
pt = Point2D(0,0)
print(Point2D(1,3).distance_from(pt))
rect = Rectangle2D(Point2D(1,1),3,4)
rect.translate(2, 1)
print(rect.get_area())
```

שינוי התנהגות של אובייקטים

```
pt1 = Point2D(0,0)  
print(pt1)
```

```
<__main__.Point2D object at 0x027C4BD0>
```

רוצים הדפסה יפה יותר

```

class Point2D:
    def __init__(self, x=0, y=0):
        self.__x = x
        self.__y = y

    def __str__(self):
        return "(" + str(self.__x) + \
            ", " + str(self.__y) + ")"

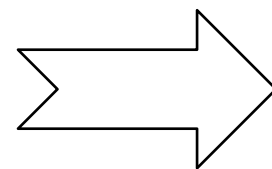
```

פייתון קורא למתודה `__str__` בכל פעולה שדורשת המרה למחרוזת

```

pt1 = Point2D(0,0)
print(pt1)
print(str(pt1))

```



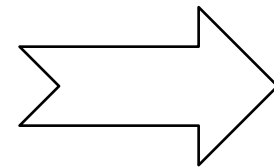
```

(0,0)
(0,0)

```

הגדרת ההתנהגות של אופרטורים

```
pt1 = Point2D(0,0)  
pt2 = Point2D(0,0)  
print(pt1==pt2)
```



False

ברירת המחדל של האופרטור == היא בדיקה שזה אותו אובייקט (כמו is) ולא השוואה של השדות.

- פייתון משתמש במתודה `__eq__` כדי לחשב את התוצאה של `==`

```
def __eq__(self, other):  
    return self.__x == other.__x and \  
           self.__y == other.__y
```

אופרטורים נוספים

- יש הרבה מתודות פלא (magic methods) נוספות שאפשר להגדיר ולשנות את ההתנהגות של הקוד:

+	object.__add__(self, other)
-	object.__sub__(self, other)
*	object.__mul__(self, other)
//	object.__floordiv__(self, other)
/	object.__div__(self, other)
%	object.__mod__(self, other)
**	object.__pow__(self, other[, modulo])
<<	object.__lshift__(self, other)
>>	object.__rshift__(self, other)
&	object.__and__(self, other)
^	object.__xor__(self, other)
	object.__or__(self, other)

<	object.__lt__(self, other)
<=	object.__le__(self, other)
==	object.__eq__(self, other)
!=	object.__ne__(self, other)
>=	object.__ge__(self, other)
>	object.__gt__(self, other)

- יש עוד ...

דוגמה

- המחלקה Set היא מיכל שמחזיק עותק יחיד של כל אובייקט שהוא מכיל
- נניח שהספרייה של פייתון לא היתה כוללת מחלקה כזו.
- איך אפשר לממש אותה?

```
class MySet:
    def __init__(self):
        self.__items=[]

    def add_item(self,item):
        if item not in self.__items:
            self.__items.append(item)

    def remove_item(self, item):
        if item in self.__items:
            self.__items.remove(item)

    def add_set(self,other):
        for item in other.__items:
            self.add_item(item)
```

הערה: זה לא מימוש יעיל!

```
def __contains__(self,item):  
    return item in self.__items
```

מגדיר את הפעולה של האופרטור in

```
def __eq__(self,other):  
    if len(self.__items) == len(other.__items):  
        result = True  
        for item in self.__items:  
            result = result and item in other.__items  
        return result  
    else:  
        return False
```

מגדיר את הפעולה של האופרטור ==

```
def __add__(self,other):  
    result = MySet()  
    result.__items = self.__items[:]  
    result.add_set(other)  
    return result
```

מגדיר את הפעולה של האופרטור +

```
def __str__(self):  
    return "{"+",".join([str(s) for s in self.__items])+"}"
```

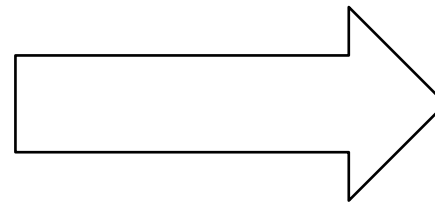
זו ההמרה שמתבצעת על ידי פונקצית המערכת str()

```
def __len__(self):  
    return len(self.__items)
```

מגדיר את התוצאה של פונקצית המערכת len()

```
set1 = MySet()  
set1.add_item(2)  
set1.add_item(4)  
set2 = MySet()  
set2.add_item(4)  
set2.add_item(3)  
set2.add_item(5)  
set1.remove_item(4)
```

```
print(set1)  
print(set2)  
print(set1+set2)  
print(2 in set1)  
print(20 in set1)  
print(len(set1))
```



```
{2}  
{4, 3, 5}  
{2, 4, 3, 5}  
True  
False  
1
```

עוד דוגמה

```
class Knapsack:
    def __init__(self, max_weight, max_volume):
        self.__max_volume = max_volume
        self.__max_weight = max_weight
        self.__weight = 0
        self.__volume = 0
        self.__items = set()

    def get_total_weight(self):
        return self.__weight

    def get_total_volume(self):
        return self.__volume
```

```
def add_item(self, item):  
    weight = item.get_weight()  
    volume = item.get_volume()  
    if self.__weight + weight <= self.__max_weight and \  
        self.__volume + volume <= self.__max_volume and \  
        item not in self.__items:  
        self.__volume += volume  
        self.__weight += weight  
        self.__items.add(item)  
        return True  
    else:  
        return False
```

מה ההנחות שלנו לגבי הפריטים שמכניסים לתרמיל?

```
def clear(self):  
    self.__weight = 0  
    self.__volume = 0  
    self.__items = set()  
  
def remove_random_item(self):  
    if len(self.__items) == 0:  
        return None  
    item = random.sample(self.__items, 1)[0]  
    self.__items.remove(item)  
    self.__weight -= item.get_weight()  
    self.__volume -= item.get_volume()  
    return item
```

```
k = Knapsack(55,10)
print(k.add_item(Book()))
print(k.get_total_volume())
print(k.get_total_weight())
print(k.add_item(Brick()))
print(k.add_item(Brick()))
print(k.remove_random_item())
print(k.remove_random_item())
print(k.remove_random_item())
```

```
class Book:
    def get_weight(self): return 5
    def get_volume(self): return 1

class Brick:
    def get_weight(self): return 50
    def get_volume(self): return 1
```

```
True
1
5
True
False
<__main__.Brick object at 0x00000000002B730F0>
<__main__.Book object at 0x00000000002B6B588>
None
```

(ההיסטוריה של) העולם על פי מל ברוקס ...

<https://www.youtube.com/watch?v=b5VsYT5vbWE>